SEMESTER S5

MICROCONTROLLERS AND EMBEDDED SYSTEMS

Dr. MAYADEVI N CET

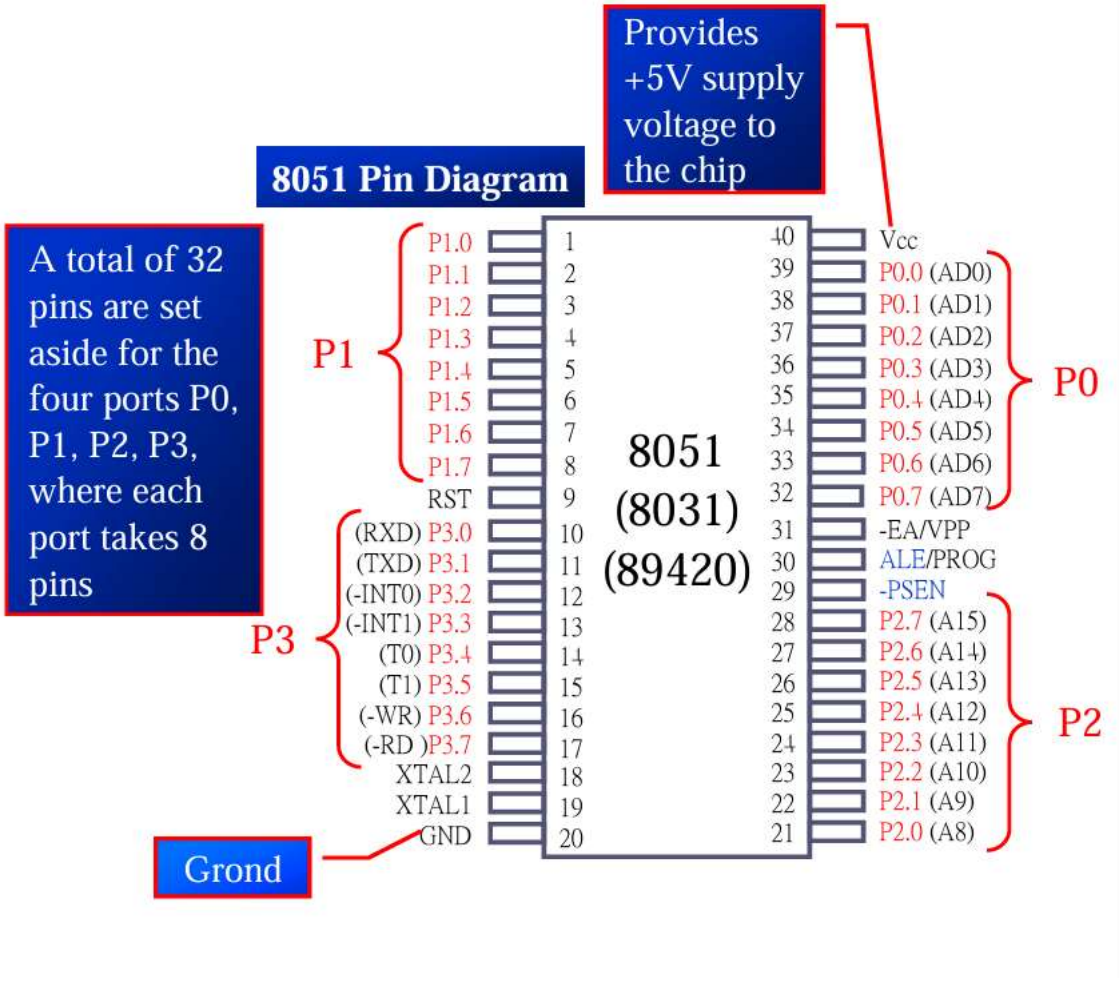
Syllabus

Module No.	Syllabus Description	Contact Hours
1	**Introduction to Microprocessors and Microcontrollers**-comparison of microprocessors and microcontrollers-Introduction to Embedded Systems-Application domain of embedded systems, features and characteristics, hard and soft real time systems **8051-Microcontrollers Hardware:** Microcontroller Architecture: IO Port structure, Register organization, general purpose RAM, Bit Addressable RAM, Special Function Registers (SFRs).	9
2	**Assembly programming of 8051:** Introduction to8051 assembly programming, Data types and Assembler directives, 8051 Addressing Modes, simple Assembly language programs(data transfer and arithmetic operations only) **8051 programming in C:** Data types and time delay in 8051, I/O	11

Syllabus

	programming in 8051 **8051 Timer/Counter programming in embedded C:** Programming 8051 timers, Counter programming,	
3	**8051 serial port programming in embedded C:** Basics of serial communication, 8051 connections to RS232, serial port programming in 8051. **8051 Interrupt programming in embedded C:** 8051 interrupts, external hardware and serial communication interrupt, Interrupt priority in 8051, Interrupt programming in C. **Interfacing:** LCD, ADC & DAC. Motor control: Relays and, stepper motor interfacing, DC motor interfacing and PWM using 8051	**12**
4	**Introduction to ARM processors** –ARM core-ARM Microcontroller-RISC vs CISC-Advanced features of ARM-Architecture versions-ARM Architecture-Instruction set architecture, operating modes, register set, General purpose registers- mode switching, conditional flags, Simple ALP programs on Arithmetic & logical operation, addition, subtraction, multiplication, division and factorial.	**12**

8051 PORTs


Provides +5V supply voltage to the chip
8051 Pin Diagram
A total of 32 pins are set aside for the four ports P0, P1, P2, P3, where each port takes 8 pins
P1
P3
P0
P2
Grond
8051
(8031)
(89420)
P1.0 1
P1.1 2
P1.2 3
P1.3 4
P1.4 5
P1.5 6
P1.6 7
P1.7 8
RST 9
(RXD) P3.0 10
(TXD) P3.1 11
(-INT0) P3.2 12
(-INT1) P3.3 13
(T0) P3.4 14
(T1) P3.5 15
(-WR) P3.6 16
(-RD) P3.7 17
XTAL2 18
XTAL1 19
GND 20
40 Vcc
39 P0.0 (AD0)
38 P0.1 (AD1)
37 P0.2 (AD2)
36 P0.3 (AD3)
35 P0.4 (AD4)
34 P0.5 (AD5)
33 P0.6 (AD6)
32 P0.7 (AD7)
31 -EA/VPP
30 ALE/PROG
29 -PSEN
28 P2.7 (A15)
27 P2.6 (A14)
26 P2.5 (A13)
25 P2.4 (A12)
24 P2.3 (A11)
23 P2.2 (A10)
22 P2.1 (A9)
21 P2.0 (A8)

8051 I/O Port Organization

- 8051 has four 8-bit bidirectional ports (P0-P3), totaling 32 I/O pins
- • P0: No pull-up, AD0–AD7
- • P1: General I/O
- • P2: A8–A15
- • P3: Alternate functions
- **Applications**
- General-purpose I/O –Push Button/Switch Interfacing (Input), Realy,Buzzer,LED etc
- External RAM/ROM interfacing

Input and Output Operation

- All the ports upon RESET are configured as input, ready to be used as input ports
- When the first 0 is written to a port, it becomes an output
- To reconfigure it as an input, a 1 must be sent to the port
- To use any of these ports as an input port, it must be programmed

OUTPUT -PROGRAMMING

WAP to send 55 H and AA H continuously to port A

The following code will continuously send out to port 0 the alternating value 55H and AAH

```
BACK:   MOV     A , #55H
        MOV     PO , A
        ACALL   DELAY
        MOV     A , #OAAH
        MOV     PO , A
        ACALL   DELAY
        SJMP    BACK
```

Write an assembly language program for the (8051) to continuously toggle all the bits of Port 1 (P1) after a short delay.

- **Algorithm**
- Initialize Port 1.
- Complement (toggle) all bits of Port 1.
- Call a short delay subroutine.
- Repeat the process forever.

Write an assembly language program for the DS89C420 (8051) to continuously toggle all the bits of Port 1 (P1) after a short delay

```
        ORG 0000H
        MOV P1,#00H
BACK:   CPL P1
        ACALL DELAY
        SJMP
DELAY:
        MOV R7,#100
   L1:  DJNZ R7,L1
   `    RET
```

8051 DELAY CACLULATION

Program:

```
DELAY:  MOV R7,#100
L1:     DJNZ R7,L1
        RET
```

Assumptions:

- 8051 (AT89S51)
- Crystal Frequency = 11.0592 MHz
- 1 Machine Cycle = 12 Clock Cycles

Machine Cycle Time

= 12 / 11.0592 MHz

= $12 / 11.0592 \times 10^6$

= 1.085 µs

Step 1: Machine Cycle Time

1 Machine Cycle = 12 Clock Cycles → 1 Machine Cycle Time (T_{MC}) = 1.085 µs

Step 2: Instruction Execution Time

Instruction	Machine Cycles	Time per Execution	Number of Executions	Total Time
MOV R7,#100	1	1 × 1.085 µs = 1.085 µs	1	1.085 µs
DJNZ R7,L1	2	2 × 1.085 µs = 2.17 µs	100	100 × 2.17 µs = 217 µs
RET	2	2 × 1.085 µs = 2.17 µs	1	2.17 µs

Step 3: Total Delay Calculation

Time for MOV	=	1.085 µs
Time for DJNZ (100 times)	=	217 µs
Time for RET	=	2.17 µs
Total Delay	=	**220.255 µs**
		≈ 220 µs (0.22 ms)

Summary Table

Instruction	Count	Time
MOV R7,#100	1	1.085 µs
DJNZ R7,L1	100	217 µs
RET	1	2.17 µs
Total Delay		≈ 220 µs

OUT PUT-PROGRAMMING

Sometimes we need to access only 1 or 2 bits of the port

```
BACK:   CPL     P1.2            ;complement P1.2
        ACALL   DELAY
        SJMP    BACK

;another variation of the above program
AGAIN:  SETB    P1.2            ;set only P1.2
        ACALL   DELAY
        CLR     P1.2            ;clear only P1.2
        ACALL   DELAY
        SJMP    AGAIN
```

P0	P1	P2	P3	Port Bit
P0.0	P1.0	P2.0	P3.0	D0
P0.1	P1.1	P2.1	P3.1	D1
P0.2	P1.2	P2.2	P3.2	D2
P0.3	P1.3	P2.3	P3.3	D3
P0.4	P1.4	P2.4	P3.4	D4
P0.5	P1.5	P2.5	P3.5	D5
P0.6	P1.6	P2.6	P3.6	D6
P0.7	P1.7	P2.7	P3.7	D7

BIT Level Instructions

- Instructions that are used for signal-bit operations are as following

Single-Bit Instructions

Instruction	Function
SETB bit	Set the bit (bit = 1)
CLR bit	Clear the bit (bit = 0)
CPL bit	Complement the bit (bit = NOT bit)
JB bit, target	Jump to target if bit = 1 (jump if bit)
JNB bit, target	Jump to target if bit = 0 (jump if no bit)
JBC bit, target	Jump to target if bit = 1, clear bit (jump if bit, then clear)

OUT PUT-PROGRAMMING

Write the following programs.
Create a square wave of 50% duty cycle on bit 0 of port 1.

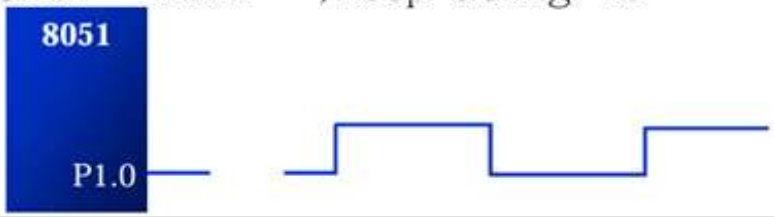
Solution:

The 50% duty cycle means that the "on" and "off" state (or the high and low portion of the pulse) have the same length. Therefore, we toggle P1.0 with a time delay in between each state.

```
HERE:   SETB    P1.0    ;set to high bit 0 of port 1
        LCALL   DELAY   ;call the delay subroutine
        CLR     P1.0    ;P1.0=0
        LCALL   DELAY
        SJMP    HERE    ;keep doing it
```

Another way to write the above program is:

```
HERE:   CPL     P1.0    ;set to high bit 0 of port 1
        LCALL   DELAY   ;call the delay subroutine
        SJMP    HERE    ;keep doing it
```



IN PUT-PROGRAMMING

To make port 1 an input port, it must be programmed as such by writing 1 to all its bits

Port 1 is configured first as an input port by writing 1s to it, then data is received from that port and saved in R7 and R5

```
MOV     A,#0FFH     ;A=FF hex
MOV     P1,A        ;make P1 an input port
                    ;by writing it all 1s
MOV     A,P1        ;get data from P1
MOV     R7,A        ;save it to in reg R7
ACALL   DELAY       ;wait
MOV     A,P1        ;another data from P1
MOV     R5,A        ;save it to in reg R5
```

Write a program to perform the following:
(a) Keep monitoring the P1.2 bit until it becomes high
(b) When P1.2 becomes high, write value 45H to port 0
(c) Send a high-to-low (H-to-L) pulse to P2.3

Solution:

```
        SETB P1.2          ;make P1.2 an input
        MOV  A,#45H        ;A=45H
AGAIN:  JNB  P1.2,AGAIN ; get out when P1.2=1
        MOV  P0,A          ;issue A to P0
        SETB P2.3          ;make P2.3 high
        CLR  P2.3          ;make P2.3 low for H-to-L
```

Example 4-4

Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a high-to-low pulse to port P1.5 to turn on a buzzer.

Solution:

```
HERE:   JNB  P2.3,HERE ;keep monitoring for high
        SETB P1.5      ;set bit P1.5=1
        CLR  P1.5      ;make high-to-low
        SJMP HERE      ;keep repeating
```